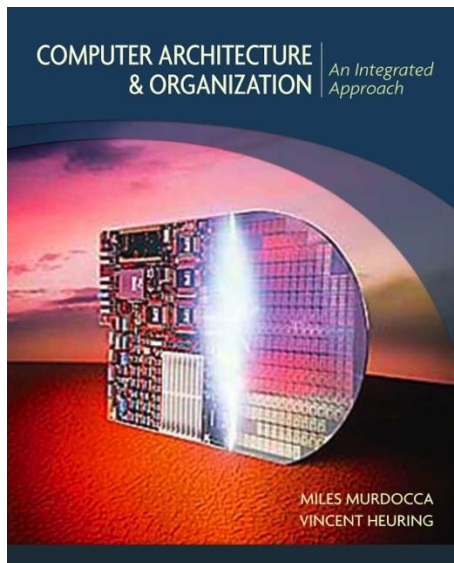


Organización de las Computadoras

Carolina Chaves Garcia



Estructuras básicas y flujo de ejecución

Estructura código assembler

Estructura código assembler

.data: Contiene datos inicializados

.bss: Contiene datos no inicializados

.text: Contiene el programa

.stack: Contiene información sobre la configuración inicial de la pila

Modos de direccionamiento

Modos de direccionamiento

Los modos de direccionamiento definen cómo una instrucción accede a los operandos en memoria o registros. Son fundamentales para optimizar código y mejorar la eficiencia del programa.

Modo inmediato:

- Operando dentro de la instrucción.
- El operando está directamente en la instrucción.

`addi x5, x0, 10` `# x5 = 10` (Carga inmediata de 10 en el registro x5)

Modo de registro:

- Los operandos están en registros.

`add x6, x5, x4` `# x6 = x5 + x4`

Modo indirecto:

- Uso de registros para apuntar a direcciones de memoria.
- El registro contiene la dirección de memoria donde se encuentra el dato.

`lw x12, 0(x11)` `# x12` obtiene el valor almacenado en la dirección contenida en `x11`

Modos de direccionamiento

Modo directo (absoluto):

- Dirección de memoria especificada.
- La instrucción contiene la dirección de memoria del dato.
- No es común en RISC-V, pero se puede simular con la (load address).

`la x10, valor    # Cargar la dirección de 'valor' en x10`

`lw x11, 0(x10) # Cargar el dato de la memoria en x11`

- `la` (Load Address - Cargar Dirección)
 - La instrucción `la` se usa para cargar la dirección de memoria de una variable en un registro.
- `lw` (Load Word - Cargar una Palabra de Memoria)
 - La instrucción `lw` se usa para leer un valor almacenado en memoria y guardarlo en un registro.

Modos de direccionamiento

Modo indexado:

- Uso de desplazamientos para acceder a posiciones de memoria.
- Útil para manejar arreglos y estructuras de datos.

```
addi x13,x0, 4      # Tamaño de un entero  
mul x14, x7, x13    # Calcula el desplazamiento (índice * 4)  
lw x15, 0(x14)      # Carga el elemento del array
```

Modo relativo al PC

- Utilizado para saltos y llamadas a funciones.

```
beq x5, x0, etiqueta    # Si x5 es 0, salta a 'etiqueta'
```

Modos de direccionamiento

Ejemplos

Ejemplo 1:

- Definir un valor en memoria:

`.data`

`numero: .word 25   # Definimos una variable en memoria con valor 25`

Ejemplo 2

- Cargarlo en un registro usando la y lw:

`.text`

`la x10, numero   # Carga la dirección de 'numero' en x10`

`lw x11, 0(x10)   # Carga el valor almacenado en 'numero' en x11. x11 ahora tiene 25`

Ejemplo 3

- Sumar un valor inmediato y almacenar en memoria:

`addi x11, x11, 5   # x11 ahora es 30`

`sw x11, 0(x10)   # Guarda el resultado en memoria`

Organización de la memoria

Organización de la memoria

Segmento	Descripción	Contenido Ejemplo
Código (Text)	Instrucciones del programa (binario)	Código máquina de las funciones
Datos (Data)	Variables globales e inicializadas	int x = 10; (global)
Heap	Memoria dinámica (allocada en tiempo de ejecución) Crece hacia arriba	malloc(), new en C/C++
Stack	Memoria para llamadas a funciones y variables locales Almacena datos temporales Crece hacia abajo	Argumentos, direcciones de retorno, variables locales

Control de flujo en ensamblador

Control de flujo en ensamblador

- El control de flujo permite modificar la ejecución del programa según condiciones o repeticiones.
- Se usan instrucciones de salto (branching) y bucles (loops) para tomar decisiones y repetir tareas.

Saltos condicionales e incondicionales (JMP, JNZ, JZ, JG, JL, JE, JNE):

- En ensamblador, los saltos permiten cambiar la ejecución del programa dependiendo de valores en los registros o banderas de estado.
- Saltos incondicionales **JMP**
 - Salta a una etiqueta sin evaluar condiciones.

JMP etiqueta # Salta a "etiqueta"

JMP fin

add x5, x5, x6 # Esta línea nunca se ejecutar

fin:

Control de flujo en ensamblador

- Salto condicionales
 - Dependen del resultado de una comparación.

Instrucción	Significado
JZ	Salta si el valor es 0 (Zero)
JNZ	Salta si el valor no es 0
JE	Salta si dos valores son iguales
JNE	Salta si dos valores no son iguales
JG	Salta si es mayor (> en C)
JL	Salta si es menor (< en C)

Control de flujo en ensamblador

Ejemplos

Ejemplo 1:

- Comparar dos valores. ¿Cuál es mayor 10 o 20?:

```
addi x5, x0, 10    # Cargar 10 en x5
addi x6, x0, 20    # Cargar 20 en x6
bge x5, x6, mayor  # Si x5 >= x6, salta a "mayor"
addi x7, x0, 1     # Si no se cumple, x7 = 1
j fin              # Salta a "fin"
mayor:
    addi x7,x0, 2   # Si x5 >= x6, x7 = 2
fin:
```

Control de flujo en ensamblador

Ejemplos

Ejemplo 2:

- Evaluar si un número es positivo o negativo, numero = -10:

addi x5,x0, -10 # Cargar un número negativo en x5

bge x5, x0, positivo # Si x5 >= 0, salta a "positivo"

Si x5 es negativo

addi x6,x0, 0 # Guardar 0 en x6 (indica negativo)

j fin

positivo:

addi x6,x0, 1 # Guardar 1 en x6 (indica positivo)

fin:

Control de flujo en ensamblador

Bucles

- Permiten repetir instrucciones.
- **Bucle FOR**
 - Se basa en un contador con una condición de salida.
 - **Ejemplo:** Sumar los primeros 5 números naturales

```
addi x5, x0, 0  # Acumulador suma = 0
addi x6, x0, 1  # Contador i = 1
addi x7, x0, 5  # Límite del bucle (5)
for_loop:
    add x5, x5, x6 # suma = suma + i
    addi x6, x6, 1 # i++
    bge x6, x7, fin # Si i >= 5, salir
    j for_loop     # Repetir (salto incondicional para evitar confusión)
fin:
```


Control de flujo en ensamblador

Bucles

- **Bucle WHILE**

- Se ejecuta mientras una condición sea verdadera.
- **Ejemplo:** Contar hasta que un número (10) llegue a 0

```
addi x5,x0, 10 # x5 = 10
```

```
while_loop:
```

```
    beq x5, x0, fin # Si x5 == 0, salir
```

```
    addi x5, x5, -1 # x5--
```

```
    j while_loop # Repetir
```

```
fin:
```

Control de flujo en ensamblador

Bucles

- **Bucle LOOP**

- Usa un registro como contador y se ejecuta hasta que llega a 0.
- **Ejemplo:** Imprimir una cadena N veces (pseudocódigo)

addi x5,x0, 5 # x5 = 5 (contador)

loop:

(Aquí iría la rutina de impresión)

addi x5, x5, -1 # Decrementar contador

bne x5,x0, loop # Si x5 ≠ 0, repetir

fin:

Manejo de pila y subrutinas

Manejo de pila y subrutinas

El Stack (pila)

- Se usa para guardar valores temporales.
- En RISC-V, el stack crece hacia direcciones menores (decrementa al hacer push). = Crece hacia abajo en la memoria.
- sp (Stack Pointer) apunta a la cima de la pila.
- El stack es una estructura LIFO (Last In, First Out) que almacena:
 - Variables locales de funciones.
 - Argumentos pasados a funciones.
 - Direcciones de retorno (para saber dónde continuar después de una llamada a función).
 - Valores temporales (como registros que deben preservarse).

Manejo de pila y subrutinas

El Stack (pila)

- Instrucciones PUSH y POP en RISC-V
 - RISC-V NO tiene instrucciones PUSH y POP nativas.
 - Se implementan manualmente con sw y lw.
 - Guardar un valor en la pila (PUSH)
 - `addi sp, sp, -4 # Reservar espacio en la pila (4 bytes)`
 - `sw x5, 0(sp) # Guardar x5 en la pila`
 - Sacar un valor de la pila (POP)
 - `lw x5, 0(sp) # Cargar el valor de la pila en x5`
 - `addi sp, sp, 4 # Liberar espacio en la pila`

Manejo de pila y subrutinas

El Stack (pila)

- Ejemplo: Guardar dos valores en la pila y recuperarlos

```
addi x5,x0, 10      # x5 = 10
addi x6,x0, 20      # x6 = 20
# PUSH
addi sp, sp, -8     # Reservar espacio (dos valores)
sw x5, 4(sp)        # Guardar x5
sw x6, 0(sp)        # Guardar x6
# POP
lw x6, 0(sp)        # Recuperar en x6
lw x5, 4(sp)        # Recuperar en x5
addi sp, sp, 8      # Restaurar pila
```

Manejo de pila y subrutinas

Llamadas a subrutinas

- Permiten reutilizar código y modularizar el programa.

Instrucción	Función
jal etiqueta	Llamar a una función (Jump And Link)
jalr x1	Retornar de una función (Jump And Link Register)
ret	Equivalente a jalr x1

Manejo de pila y subrutinas

Llamadas a subrutinas

- **Ejemplo:** Implementación de una función que reciba dos números y retorne la suma. = Subrutina que suma dos números

.text

main:

addi x10,x0, 5 # Primer número en a0

addi x11,x0, 10 # Segundo número en a1

jal suma # Llamar a la subrutina

El resultado queda en x10

ebreak # Detener programa

suma:

add x10, x10, x11 # x10 = x10 + x11

ret # Retornar

Manejo de pila y subrutinas

Llamadas a subrutinas

- **Paso de Parámetros con la Pila**

- Si una subrutina necesita más de dos parámetros, los extras se guardan en la pila.
- Ejemplo: Función que suma 3 números usando la pila

.text

main:

```
addi x10,x0, 5    # Primer número
addi x11,x0, 10   # Segundo número
addi x12,x0, 15   # Tercer número
# PUSH tercer parámetro
addi sp, sp, -4
sw x12, 0(sp)
jal suma_tres     # Llamar a la función
# POP después de la llamada
addi sp, sp, 4
ebreak
```

Manejo de pila y subrutinas

Llamadas a subrutinas

suma_tres:

lw x5, 0(sp)	# Recuperar tercer parámetro
add x10, x10, x11	# Sumar x10 + x11
add x10, x10, x5	# Sumar resultado + x5
ret	# Retornar

EJERCICIO

- Implementación de una secuencia condicional
- Escribir un programa en ensamblador que:
 - Pida un número al usuario (simulado con un valor fijo en memoria).
 - Verifique si el número es par o impar.
 - Si es par, almacene 1 en una dirección de memoria; si es impar, almacene 0.

EJERCICIO

.data

numero: .word 7 # Simulamos entrada del usuario con un número fijo
resultado: .word 0 # Variable para almacenar 1 si es par, 0 si es impar

.text

main:

 # Cargar el número desde memoria

 la x10, numero # Cargar la dirección del número

 lw x11, 0(x10) # Cargar el número en x11

 # Comprobar si es par o impar usando AND con 1

 andi x5, x11, 1 # x5 = x11 & 1

 # Si x5 == 0 (par), saltar a etiqueta 'es_par'

 beq x5, zero, es_par

EJERCICIO

```
# Si no es par, es impar
li x12, 0      # Guardar 0 en x12 (impar)
j guardar_resultado
es_par:
    addi x12,x0, 1      # Guardar 1 en a2 (par)
guardar_resultado:
    la x13, resultado # Cargar la dirección de la variable resultado
    sw x12, 0(x13)     # Almacenar el resultado en memoria
    ebreak            # Fin del programa
```

Gracias